

- EXAMEN DE 1^{ère} SESSION -

NOM (en majuscules) :	Date : avril 2018
Prénom :	Section / Année :
Local :	

UE : Principes d'algorithmique et programmation – C#

Epreuve : Examen final

Professeur : François Koeune

-
- Matériel autorisé : ordinateur fixe de l'EPHEC (sans accès internet), Visual Studio et accès à la documentation MSDN.
 - Durée : 2h30.
-

Ne pas oublier :

- Lisez attentivement les questions avant de répondre.
- Toutes les réponses doivent être stockées sous forme de projets (dont les noms sont précisés ci-dessous) sur votre lecteur G:, où je pourrai les récupérer.
- Si vous le désirez, vous pouvez me demander de vérifier que vos réponses sont bien stockées au bon endroit, à condition de le faire avant la fin de l'examen.
- Si vous avez des difficultés pour accéder à la documentation MSDN, effectuez l'opération décrite en dernière page.

Première partie, sans utiliser d'ordinateur

Question 1.

Ecrivez **exactement** ce qui s'affichera à la console si l'on exécute le programme suivant :

```
static void Main(string[] args)
{
    int i = 6;
    while (i < 40)
    {
        if (i == 11) Console.WriteLine("Gagné!");
        Console.WriteLine(i);
        i = i + 10;
    }
}
```

Remarques:

- Ce programme ne contient pas d'erreurs
- Inutile de rédiger un texte expliquant ce que le programme fait, indiquez juste le résultat affiché.

Réponse :

Question 2.

Expliquez ce que fait la fonction ci-dessous, qui ne contient pas d'erreur :

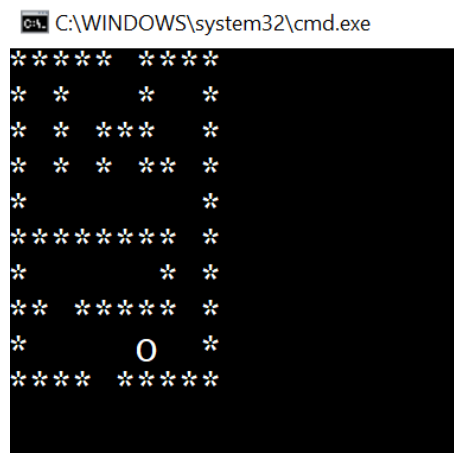
```
static int FonctionMystere(int Nb1, int Nb2)
{
    if (Nb1 < Nb2) return -1;
    else if (Nb1 > Nb2) return 1;
    else return 0;
}
```

Remarque : vous ne devez pas décrire ligne par ligne ce que fait le code, mais décrire l'effet global de la fonction (par exemple « cette fonction affiche les nombres de 1 à 10 »).

Réponse :

Deuxième partie, sur PC

L'objectif des questions suivantes est de programmer un petit jeu de labyrinthe. Au lancement, le programme affiche un labyrinthe (la génération du labyrinthe est déjà faite pour vous) et l'utilisateur peut piloter un personnage de l'entrée jusqu'à la sortie. Chacune des questions vise à écrire un élément contribuant à la solution. La dernière question vise à démontrer leur utilisation. Lorsque c'est possible, vous devez utiliser pour une réponse les fonctions écrites pour les réponses précédentes.



Si vous ne pouvez pas répondre à une des questions, vous pouvez néanmoins indiquer la signature qu'aurait la fonction correspondante, par exemple

`int Tartempion(string Nom, char Initiale)`

et utiliser cette fonction pour la suite de l'examen : ceci permettra de valider vos autres réponses.

Représentation en mémoire

Nous représenterons le labyrinthe par un tableau d'entiers de taille 10 sur 10. La première coordonnée correspond à la position verticale (comptée à partir de 0, de haut en bas) et la deuxième à la position horizontale (comptée à partir de 0, de gauche à droite). Les cases vides du labyrinthe seront notées avec la valeur 0 et les murs avec la valeur 1.

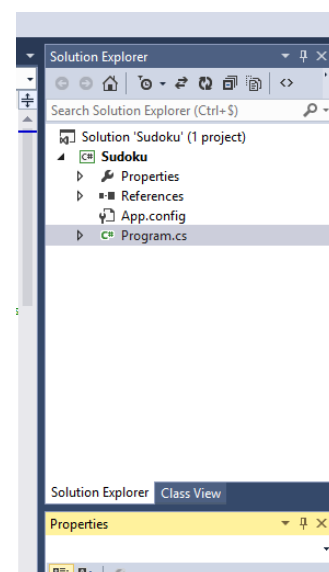
La position du joueur sera codée sous la forme de deux entiers, PosV et PosH, pour ses positions verticale et horizontale.

L'entrée du labyrinthe se trouve toujours en bas, et toujours à la position (9, 4). La position exacte de la sortie peut varier, mais elle est toujours en haut du labyrinthe, donc à la ligne 0.

Le joueur pilotera son personnage au moyen des touches 'Z', 'Q', 'S', 'D'. Pour simplifier, nous supposons que les touches tapées sont toujours en majuscules.

Projet de départ

- Récupérez le projet "Labyrinthe", disponible dans le répertoire "K:\IN1-Principes algo. et programm" et copiez-le sur votre lecteur "G:". Ce projet vous servira de base de travail. C'est également là que vous écrirez vos réponses aux questions ci-dessous.
 - Remarque : si le code n'apparaît pas lorsque vous ouvrez le projet avec Visual Studio, cliquez sur "Program.cs" dans votre explorateur de solutions (cf. figure ci-contre).
- Ce projet comporte déjà les deux fonctions suivantes (il n'est pas nécessaire de comprendre comment ces fonctions sont implémentées, il vous suffit de les appeler aux endroits appropriés) :
 - Une fonction `ChargerLabyrinthe()`, qui reçoit en paramètre un numéro de labyrinthe (de 0 à 4) et charge le labyrinthe de test correspondant, qui est renvoyé sous la forme d'un tableau.



- Une fonction LireTouche() qui lit directement une touche tapée au clavier, sans que l'utilisateur doive taper <enter>. La fonction renvoie cette touche sous la forme d'un char. Notez que la lettre tapée n'est pas affichée à la console, ce qui améliorera l'aspect visuel du jeu.
- Vous trouverez également dans le répertoire "K:\IN1-Principes algo. et programm" un exécutable « Demo.exe ». Ce programme, que vous pouvez lancer en le double-cliquant, vous montre l'objectif à atteindre.

Questions (suite)

Question 3.

Ecrivez une fonction `AfficherLabyrinthe()` qui reçoit un tableau représentant un labyrinthe et l'affiche à la console. Les cases vides sont représentées par des espaces et les murs par des '*' (il ne s'agit donc pas d'afficher des 0 et des 1).

Question 4.

Ecrivez une fonction `Gagne()`. Cette fonction :

- Reçoit la position verticale et horizontale du joueur.
- Renvoie un résultat (du type le plus approprié) indiquant si le joueur a gagné. Le joueur a gagné s'il se trouve en haut du labyrinthe.

Question 5.

Ecrivez une fonction `PositionAutorisee()`. Cette fonction :

- Reçoit 3 paramètres : une position de joueur et un labyrinthe.
- Renvoie un résultat (du type le plus approprié) indiquant si la position est autorisée dans le labyrinthe, c'est-à-dire si elle est bien dans les limites du labyrinthe et ne correspond pas à un mur.

Question 6.

Ecrivez une fonction `Deplacer()`. Cette fonction :

- Reçoit 4 paramètres :
 - une position ;
 - une direction, sous la forme d'un caractère qui vaut 'Z' (haut), 'Q' (gauche), 'S' (bas) ou 'D' (droite) ;
 - un labyrinthe.
- Si le mouvement est possible, c'est-à-dire si la position d'arrivée est autorisée, la fonction modifie les paramètres de la position selon la direction. Par exemple, si la position reçue est (4, 3), la direction 'Z' et qu'un mouvement vers le haut est possible, les paramètres de la position doivent prendre après l'appel de la fonction les valeurs (3, 3).
- Dans tous les autres cas, la fonction ne fait rien.

Question 7.

Ecrivez une fonction `AfficherPerso()`, qui reçoit une position de joueur et le dessine à cette position dans le labyrinthe, sous la forme du caractère 'O'.

Vous aurez besoin de la fonction `Console.SetCursorPosition()`. Si nécessaire, la doc MSDN de cette fonction est imprimée en dernière page de ce questionnaire.

Question 8.

Ecrivez une fonction `EffacerPerso()`, qui reçoit une position de joueur et l'efface à cette position dans le labyrinthe, en le remplaçant par un espace.

Question 9.

Ecrivez une fonction `Jeu()` qui utilise les fonctions ci-dessus pour réaliser l'ensemble du jeu. Plus précisément, cette fonction :

- Reçoit en paramètre un entier entre 0 et 4 correspondant au labyrinthe à charger. Vous pouvez supposer que l'entier reçu a une valeur correcte.
- Charge et affiche le labyrinthe correspondant.
- Affiche le personnage à la position de départ. Pour rappel, celle-ci est toujours (9, 4).
- Permet au joueur de piloter le personnage dans le labyrinthe, sans traverser les murs.
- Affiche « Gagné ! » dès que le personnage atteint la sortie.

Enfin, appelez votre jeu depuis la fonction `Main()`. N'oubliez pas de vérifier qu'il fonctionne avec des labyrinthes différents.

Pour les deux dernières questions, qui ne sont pas liées au jeu du labyrinthe, récupérez le projet "Restaurant", disponible dans le répertoire "K:\IN1-Principes algo. et programm" et copiez-le sur votre lecteur "G:".

Ce programme très basique met en œuvre une application dans laquelle les clients d'un restaurant peuvent choisir entre différents menus. Au lancement, un tableau contenant les différents menus (un menu par ligne, un plat par colonne) est chargé depuis un fichier. Ensuite, l'utilisateur est invité à taper un nombre entre 0 et 5, et le menu correspondant est affiché. En d'autres termes, l'utilisateur tape un nombre, et la ligne du tableau correspondant à ce nombre est affichée (par exemple, si l'utilisateur tape 2, le menu « Mousseux, Terrine, Potage aux Tomates, Steak, Mousse au chocolat, Café » est affiché). Pour simplifier, nous supposons que l'utilisateur entre toujours une réponse valide.

La majeure partie du programme est déjà écrite, mais contient des erreurs, et l'une des fonctions manque.

Question 10.

Ecrivez (à l'endroit indiqué dans le code) la fonction `ExtraireLigne()`. Cette fonction reçoit en paramètres :

- `NumLigne` : un numéro de ligne,
- `Menu` : un tableau de string de 5 lignes et 6 colonnes,

et renvoie, sous la forme d'une liste de string, le contenu de la ligne `NumLigne` du `Menu`.

La fonction `Main()` contient déjà un appel à votre fonction. Cet appel est correct et ne doit pas être modifié.

Question 11.

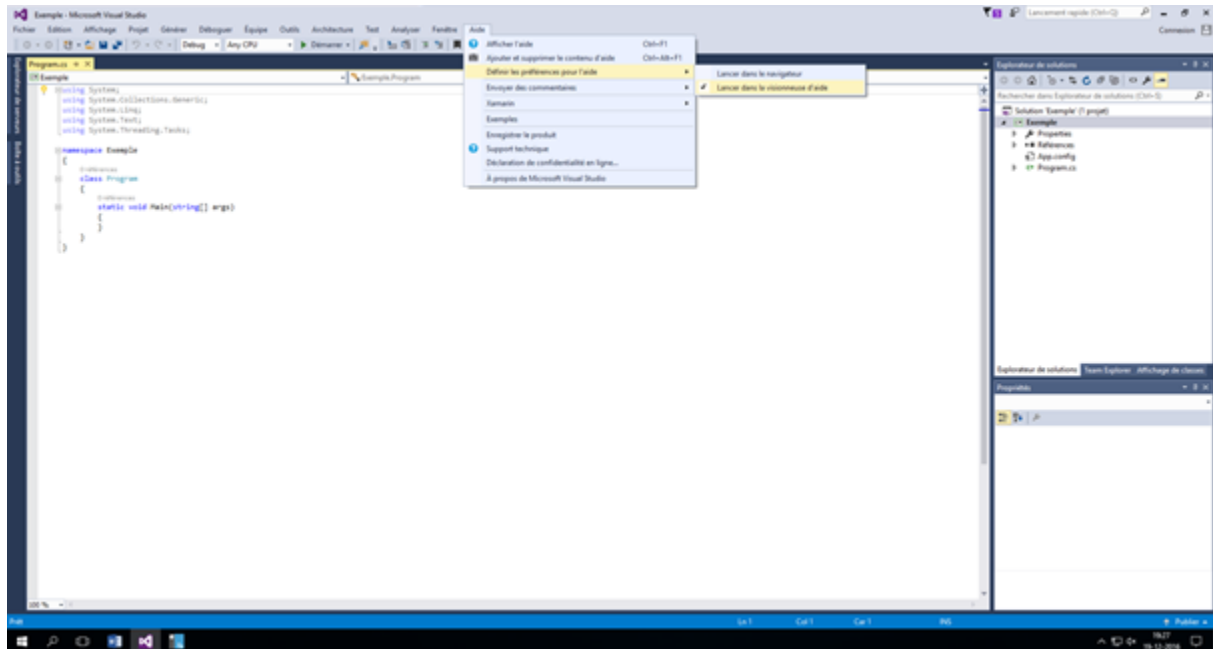
Ce programme comporte cependant plusieurs erreurs. Corrigez-le en vous aidant des outils de debugging.

Remarques :

- *La fonction `ChargerMenu()` ne contient pas d'erreurs.*
- *Il y a 4 erreurs dans le code.*

En cas de problème pour accéder à la documentation

Si vous rencontrez des difficultés pour accéder à la documentation MSDN, vérifiez que l'option des préférences pour l'aide est bien sur « lancer dans la visionneuse d'aide » et pas sur « lancer dans le navigateur ».



Console.SetCursorPosition(Int32, Int32) Method

Espace de noms: [System](#)

Assemblies: System.Console.dll, mscorlib.dll, netstandard.dll

Définit la position du curseur.

C#

 Copier

```
public static void SetCursorPosition (int left, int top);
```

Paramètres

left [Int32](#)

Position de colonne du curseur. Les colonnes sont numérotées de gauche à droite à partir de 0.

top [Int32](#)

Position de ligne du curseur. Les lignes sont numérotées de haut en bas à partir de 0.

Exceptions

[ArgumentOutOfRangeException](#)

`left` ou `top` est inférieur à zéro.

ou `left` est supérieur ou égal à [BufferWidth](#).

ou `top` est supérieur ou égal à [BufferHeight](#).

[SecurityException](#)

L'utilisateur n'est pas autorisé à effectuer cette action.

[IOException](#)

Une erreur d'E/S s'est produite.